

Table of Content

Title Page	1
Dedication	2
Acknowledgement	3
Table of Content	4
1.0 Introduction	5
2.0 Business Rules	7
3.0 Conceptual Design	9
4.0 SQL Scripts	15
4.1 Creating Tables	15
4.2 Inserting Data	15
4.3 Tables for the Database HMDB (Creation and Insertion)	15
5.0 Data Extraction	27
5.1 Extracting Patient Information	27
5.2 Retrieving Appointment Details	27
5.3 Accessing Doctor and Department Information	27
5.4 Extracting Visit Information	28
5.5 Viewing Billing Information	28
5.6 Extracting Medicines Information	28
6.0 Normalization Process	29
6.1 First Normal Form (1NF)	29
6.2 Second Normal Form (2NF)	31
6.3 Third Normal Form (3NF)	33
6.4 Boyce-Codd Normal Form (BCNF)	36
7.0 Additional Functionalities	39
8.0 Final Outcomes	43
Conclusion	44

1.0 Introduction

Project Title: Hospital Database Management System (HMDB)

In healthcare, efficient data management is essential to ensuring optimal patient care and operational efficiency. The Hospital Database Management System (HMDB) is an innovative solution that addresses hospital administration's complexities through an integrated data management approach. This project seeks to streamline the workflows associated with hospital management by providing a robust platform for handling diverse data sets related to patients, medical personnel, and administrative processes.

Purpose

The primary objective of the Hospital Database Management System (HMDB) is to facilitate the effective management of various workflows related to hospital operations. A comprehensive database system has become imperative as healthcare providers increasingly rely on technology to enhance patient care and improve operational efficiency. This system is intended to automate and optimize critical functions, including patient registration, appointment scheduling, and data manipulation, thereby reducing manual errors, minimizing administrative burdens, and improving overall patient experiences. We aim to provide healthcare professionals with timely access to essential information, enabling them to make informed decisions that positively impact patient outcomes. The HMDB is designed to support front-line medical staff and administrative personnel by ensuring that pertinent information is readily available, promoting better communication and collaboration within healthcare teams.

Scope

The scope of the Hospital Database Management System encompasses a wide range of functionalities that are pivotal for effective hospital management. These functionalities include:

1. *Patient Management:* The system facilitates comprehensive patient registration, allowing healthcare providers to capture vital patient information, including personal details, medical history, and contact information. This ensures that patient data is organized, accessible, and secure.
2. *Doctor Management:* The HMDB enables the management of doctor information, including details on specialities, contact information, and appointment schedules. This functionality ensures patients can be efficiently matched with the appropriate medical professionals based on their healthcare needs.
3. *Appointment Scheduling:* One of the critical features of the system is its ability to handle appointment scheduling. The HMDB allows patients to book appointments conveniently, view available time slots, and receive notifications, thereby reducing waiting times and enhancing the patient experience.
4. *Visit Management:* The system tracks patient visits, including consultations, examinations, and treatment procedures. It captures relevant data during each visit, such as vital signs, diagnoses, and prescribed medications, ensuring a comprehensive record of patient interactions with healthcare providers.

5. *Medicines and Treatment Management*: The HMDB allows for efficient management of medicines, treatments, and tests. It stores information about various medications, their dosages, associated costs, and records of treatment plans and test results, facilitating informed decision-making by healthcare professionals.

6. *Billing and Financial Management*: The system also includes features for managing billing information, ensuring that financial transactions related to patient care are accurately recorded and processed. This includes generating bills for services rendered, tracking payments, and managing insurance claims.

7. *Staff Management*: The HMDB provides tools for managing hospital staff, including nurses, administrative personnel, and support staff. It allows tracking staff schedules, roles, and contact information, ensuring that the hospital is adequately staffed and organized.

8. *Room and Resource Management*: The system helps manage hospital rooms, resources, and equipment. It tracks room availability, assigns rooms to patients, and monitors medical equipment status, ensuring optimal utilization of hospital resources.

Through this project, we aspire to create a scalable and adaptable platform that meets the evolving needs of healthcare providers and enhances the quality of care delivered to patients.

2.0 Business Rules

In designing and implementing the Hospital Database Management System (HMDB), we need to establish a set of business rules that govern the operations and interactions within the database. These rules ensure data integrity, promote efficient workflows and facilitate a seamless experience for system users. By adhering to these business rules, the HMDB will maintain a high standard of accuracy and reliability in managing hospital-related data.

1. Unique Patient Identification

Each patient must have a unique identifier, the Patient ID (Pat_ID). This ID is the primary key in the Patient_personal_info table and ensures that every patient record is distinct and easily retrievable. This uniqueness is essential to prevent data duplication and maintain patient information's integrity throughout the system.

2. Doctor Assignment to Departments

Each doctor must be associated with a specific department within the hospital. This relationship is established through the Doctor_info table, where each doctor is linked to their respective department via the Department ID (Dept_ID). This rule is critical for organizing medical staff by specialty, facilitating effective appointment scheduling, and ensuring patients are referred to the appropriate healthcare providers based on their medical needs.

3. Appointment Linking

Every appointment must link to a patient and a doctor, ensuring a clear connection between the services provided and the individuals involved. The Appointment table contains foreign keys referencing the Patient ID and Doctor ID, which establishes these relationships. This linkage is vital for tracking patient visits and maintaining accurate healthcare service records.

4. Unique Email Addresses

The system enforces a rule that each patient and doctor must have a unique email address stored in the respective tables (Patient_personal_info and Doctor_info). This uniqueness is enforced through a unique constraint on the email columns, ensuring that no two individuals can share the same email address. This rule enhances communication within the system and helps prevent confusion in patient and doctor interactions.

5. Valid Contact Numbers

All contact numbers entered into the database must adhere to specific format and validity constraints. Patient and doctor contact numbers are stored as BIGINT to accommodate long numbers and are checked for validity upon entry. This rule maintains accurate contact information, facilitating timely communication regarding appointments, treatments, and follow-ups.

6. Foreign Key References

The HMDB employs foreign key constraints to enforce relationships between different tables. For example, the Appointment table references the Patient ID and Doctor ID from the Patient_personal_info and Doctor_info tables. Similarly, the Visit table references the Appointment table. These foreign key references are crucial for maintaining referential integrity, ensuring that relationships between records are consistent and valid throughout the database.

7. Treatment and Medication Management

When recording treatments and medications in the Treatment table, each entry must reference valid and existing records in the Medicines and Visit tables. This rule ensures that only approved medications and treatment protocols are recorded, essential for patient safety and compliance with medical standards.

8. Room Assignment

Each patient admitted to the hospital must be assigned a room from the Rooms table. The Room ID serves as a foreign key in the In_Patients table, ensuring patients are allocated to available and suitable rooms. This rule aids in efficient hospital resource management and enhances patient experience.

9. Billing Information Integrity

The Billing_info table must accurately reference the corresponding patient, visit, appointment, and doctor records. This rule ensures that billing is correctly linked to services, facilitating accurate invoicing and financial management. Each bill must also be associated with a valid Staff ID, ensuring accountability in the billing process.

10. Data Validation and Constraints

The database design incorporates various validation checks and constraints to uphold data integrity. For instance, fields such as dates of birth and appointment times must be logically valid, ensuring that no future dates are mistakenly entered for past events. These validations help maintain the data's reliability and the system's overall functionality.

Establishing robust business rules is essential for the effective functioning of the Hospital Database Management System. These rules govern the integrity and consistency of the data and facilitate smooth operational workflows within the hospital. By adhering to these guidelines, the HMDB will enhance the efficiency of hospital management processes, improve patient care, and ensure compliance with healthcare regulations. Ultimately, implementing these business rules will contribute to a comprehensive and reliable database system that meets the needs of healthcare providers and patients alike.

3.0 Conceptual Design

The conceptual design phase of the Hospital Database Management System (HMDB) involves creating a high-level overview of the system's structure through an Entity-Relationship Diagram (ERD). This design outlines the entities that will be represented in the database, their attributes, and the relationships that exist between them. This ensures the database can efficiently handle various workflows related to hospital management, such as patient registration, appointment scheduling, and data manipulation. The ERD for the Hospital HMDB includes several entities that represent the critical components of hospital operations (Figure 1). Each entity is linked to one or more relationships that define how they interact. The first step in the conceptual transformation process is to identify the main entities that will represent the critical components of the hospital management system. Entities are objects or concepts that have distinct existence and can be categorized based on the information that needs to be stored. In the HMDB, the following entities have been identified:

Patient: Represents individuals seeking medical care and includes their personal information and medical history.

Doctor: Represents healthcare professionals who provide medical services, including their specialities and departmental affiliations.

Appointment: Represents scheduled visits between patients and doctors, detailing the purpose and time of the visit.

Department: Represents the various medical specialities within the hospital, such as cardiology, orthopaedics, and paediatrics.

Visit: Represents a consultation or treatment session between a patient and a healthcare provider.

Medicines: Represents pharmaceutical products prescribed to patients, including their names and dosages.

Treatment: Represents the medical procedures and medications administered during patient visits.

Rooms: Represents the physical spaces within the hospital, such as patient rooms, operation theatres, and examination rooms.

Staff: Represents personnel who support hospital operations, including nurses, administrative staff, and technicians.

In_Patients: Represents patients admitted for overnight stays or longer periods, detailing their treatments and accommodations.

Billing_info: Represents the financial transactions related to patient care, including charges for services, medications, and accommodations.

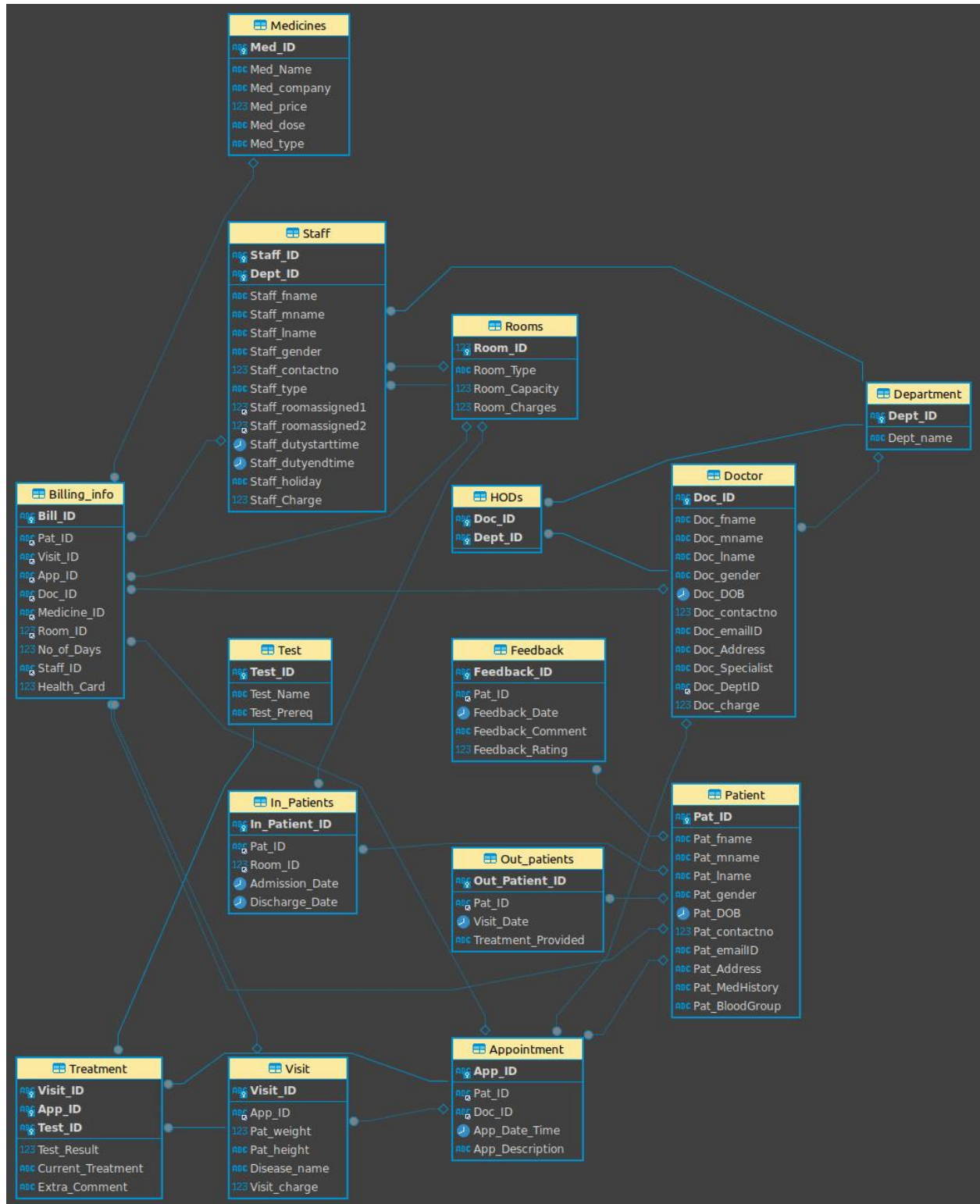


Figure 1: The E-R Diagram of the HMDB

Entities and Their Attributes

1. Patient

Attributes:

Pat_ID (Primary Key)
Pat_fname
Pat_mname
Pat_lname
Pat_gender
Pat_DOB
Pat_contactno
Pat_emailID
Pat_Address
Pat_MedHistory
Pat_BloodGroup

2. Doctor

Attributes:

Doc_ID (Primary Key)
Doc_fname
Doc_mname
Doc_lname
Doc_gender
Doc_DOB
Doc_contactno
Doc_emailID
Doc_Address
Doc_Specialist
Doc_DeptID (Foreign Key)
Doc_charge

3. Appointment

Attributes:

App_ID (Primary Key)
Pat_ID (Foreign Key)
Doc_ID (Foreign Key)
App_Date_Time
App_Description

4. Department

Attributes:

Dept_ID (Primary Key)
Dept_name

5. Visit

Attributes:

Visit_ID (Primary Key)

App_ID (Foreign Key)

Pat_weight

Pat_height

Disease_name

Visit_charge

6. Medicines

Attributes:

Med_ID (Primary Key)

Med_Name

Med_company

Med_price

Med_dose

Med_type

7. Treatment

Attributes:

Visit_ID (Foreign Key)

App_ID (Foreign Key)

Test_ID (Foreign Key)

Test_Result

Current_Treatment

Extra_Comment

Current_Med (Foreign Key)

8. Rooms

Attributes:

Room_ID (Primary Key)

Room_Type

Room_Charge

9. Staff

Attributes:

Staff_ID (Primary Key)

Staff_fname

Staff_mname

Staff_lname

Staff_gender

Staff_contactno

Staff_type

Staff_roomassigned1 (Foreign Key)

Staff_roomassigned2 (Foreign Key)

Staff_dutystarttime
Staff_dutyendtime
Staff_holiday
Staff_DeptID (Foreign Key)
Staff_Charge

10. In_Patients

Attributes:

Pat_ID (Foreign Key)
Admission_Date_Time
In_treatments
Room_assigned (Foreign Key)
OT_assigned (Foreign Key)
Operation_Date_Time

11. Billing_info

Attributes:

Bill_ID (Primary Key)
Pat_ID (Foreign Key)
Visit_ID (Foreign Key)
App_ID (Foreign Key)
Doc_ID (Foreign Key)
Medicine_ID (Foreign Key)
Room_ID (Foreign Key)
No_of_Days
Staff_ID (Foreign Key)
Health_Card

Relationships

The relationships between the entities in the HMDB are critical for defining how data flows through the system. The following outlines the key relationships:

1. Patient to Appointment:

Relationship Type: One-to-Many

A single patient can have multiple appointments, but each appointment is linked to one specific patient.

2. Doctor to Appointment:

Relationship Type: One-to-Many

A doctor can have multiple appointments with different patients, but each appointment corresponds to only one doctor.

3. Department to Doctor:

Relationship Type: One-to-Many

Each department can have multiple doctors, while each doctor belongs to only one department.

4. Appointment to Visit:

Relationship Type: One-to-One

Each appointment leads to one visit, where the details of the patient's consultation are recorded.

5. Visit to Treatment:

Relationship Type: One-to-Many

A single visit can involve multiple treatments (e.g., different tests or procedures), but each treatment is associated with one visit.

6. Medicines to Treatment:

Relationship Type: One-to-Many

A single medicine can be prescribed in multiple treatments, each involving one or more medicines.

7. Rooms to Staff:

Relationship Type: One-to-Many

A room can be assigned to multiple staff members, but each staff member can be assigned to one or two specific rooms.

8. Patient to In_Patients:

Relationship Type: One-to-One

Each patient can only be registered as an inpatient once during a specific admission, while each inpatient record corresponds to one patient.

9. Billing_info to Patient:

Relationship Type: One-to-Many

A single patient may have multiple billing records over time, but each billing record refers to one specific patient.

10. Billing_info to Visit and Appointment:

Relationship Type: Many-to-One

Each billing record can relate to one visit and one appointment, but one visit can generate multiple billing records if necessary.

This design forms the foundation for the database implementation, supporting the primary objective of enhancing the quality and efficiency of hospital operations.

4.0 SQL Scripts

The following section provides detailed SQL scripts for creating tables in the Hospital Database Management System (HMDB) and inserting data into these tables. Each table is defined with appropriate primary keys, foreign keys, and constraints to maintain data integrity and ensure proper relationships between the tables.

4.1 Creating Tables

The first step in the database design involves creating the necessary tables to store the relevant data for the hospital management system.

4.2 Inserting Data

Once the tables have been created, the next step involves populating them with data.

N.B: All our codes can be found in the supplementary materials.

4.3 Tables for the Database HMDB (Creation and Insertion)

Table structure for table Appointment

Column	Type	Null	Default
App_ID	varchar(10)	No	
Pat_ID	varchar(10)	Yes	NULL
Doc_ID	varchar(10)	Yes	NULL
App_Date_Time	datetime	Yes	NULL
App_Description	varchar(255)	Yes	NULL

Dumping data for table Appointment

A001	P001	D001	2024-08-01 09:00:00	Routine check-up
A002	P002	D002	2024-08-02 10:00:00	Consultation for asthma
A003	P003	D003	2024-08-03 11:00:00	Knee pain evaluation
A004	P004	D004	2024-08-04 14:00:00	Headache assessment
A005	P005	D005	2024-08-05 15:00:00	General health check
A006	P006	D001	2024-08-06 09:30:00	Heart check-up
A007	P007	D002	2024-08-07 10:30:00	Consultation for hypertension
A008	P008	D003	2024-08-08	Knee surgery

			11:30:00	evaluation
A009	P009	D004	2024-08-09 14:00:00	Migraine follow-up
A010	P010	D005	2024-08-10 15:00:00	Dental check-up

Table structure for table Billing_info

Column	Type	Null	Default
Bill_ID	varchar(10)	No	
Pat_ID	varchar(10)	Yes	NULL
Visit_ID	varchar(10)	Yes	NULL
App_ID	varchar(10)	Yes	NULL
Doc_ID	varchar(10)	Yes	NULL
Medicine_ID	varchar(10)	Yes	NULL
Room_ID	int	Yes	NULL
No_of_Days	int	Yes	NULL
Staff_ID	varchar(10)	Yes	NULL
Health_Card	tinyint(1)	Yes	NULL

Dumping data for table Billing_info

B001	P001	V001	A001	D001	M001	NULL	0	S006	0
B002	P002	V002	A002	D002	M002	NULL	0	S006	0
B003	P003	V003	A003	D003	M003	NULL	0	S006	0
B004	P004	V004	A004	D004	M004	NULL	0	S006	0
B005	P005	V005	A005	D005	M001	NULL	0	S006	0
B006	P006	V006	A006	D001	M002	NULL	0	S006	0
B007	P007	V007	A007	D002	M003	NULL	0	S006	0
B008	P008	V008	A008	D003	M004	NULL	0	S006	0
B009	P009	V009	A009	D004	M005	NULL	0	S006	0
B010	P010	V010	A010	D005	M006	NULL	0	S006	0

Table structure for table Department

Column	Type	Null	Default
Dept_ID	varchar(10)	No	
Dept_name	varchar(50)	Yes	NULL

Dumping data for table Department

D01	Cardiology
D02	Pediatrics
D03	Orthopedics

D04	Neurology
D05	General Medicine
D06	Dermatology
D07	Oncology
D08	Gastroenterology
D09	Ophthalmology
D10	Dentistry

Table structure for table Doctor

Column	Type	Null	Default
Doc_ID	varchar(10)	No	
Doc_fname	varchar(50)	Yes	NULL
Doc_mname	varchar(50)	Yes	NULL
Doc_lname	varchar(50)	Yes	NULL
Doc_gender	char(1)	Yes	NULL
Doc_DOB	date	Yes	NULL
Doc_contactno	bigint	Yes	NULL
Doc_emailID	varchar(100)	Yes	NULL
Doc_Address	varchar(255)	Yes	NULL
Doc_Specialist	varchar(50)	Yes	NULL
Doc_DeptID	varchar(10)	Yes	NULL
Doc_charge	int	Yes	NULL

Dumping data for table Doctor

D001	Samuel	Olu	Adesina	M	1978-03-12	7012345678	samuel.adesina@example.com	123 Hospital Rd, Lagos	Cardiologist	D01	5000
D002	Joyce	Ada	Emeka	F	1980-06-22	7023456789	joyce.emeka@example.com	456 Clinic St, Enugu	Pediatrician	D02	4500
D003	Chijiokwe	Ugo	Obinna	M	1975-12-05	7034567890	chijiokwe.obinna@example.com	789 Doctor St, Port Harcourt	Orthopedic Surgeon	D03	6000
D004	Ngozi	Chika	Nwachukwu	F	1985-09-15	7045678901	ngozi.nwachukwu@example.com	321 Health St, Lagos	Neurologist	D04	7000

							wu@ex ample.c om	Ave, Abuja			
D005	Ikenna	Chinedu	Okeke	M	1990-04-18	7056789012	ikenna.o keke@ exampl e.com	654 Medica l St, Owerri	General Practiti oner	D05	4000
D006	Chigozi	Ifesinachi	Eze	F	1983-07-22	7067890123	chigozi e.eze@ exampl e.com	222 Oak St, Lagos	Dermat ologist	D06	5000
D007	Adaobi	Ugochi	Igbokwe	F	1986-11-05	7078901234	adaobi. igbokw e@exa mple.c om	333 Pine Ave, Enugu	Oncolo gist	D07	6000
D008	Obi	Chinonso	Nnaji	M	1984-04-13	7089012345	obi.nna ji@exa mple.c om	444 Maple Rd, Port Harcou rt	Gastroe nterolo gist	D08	7000
D009	Emmanuel	Chijioku	Mbachu	M	1979-10-29	7090123456	emman uel.mb achu@ exampl e.com	555 Birch St, Abuja	Ophtha lmologi st	D09	5000
D010	Ifeoma	Chinwe	Obiora	F	1991-02-12	7101234567	ifeoma. obiora @exam ple.co m	666 Elm St, Owerri	Dentist	D10	4000

Table structure for table Feedback

Column	Type	Null	Default
Feedback_ID	varchar(10)	No	
Pat_ID	varchar(10)	Yes	NULL
Feedback_Date	date	Yes	NULL
Feedback_Comment	varchar(255)	Yes	NULL
Feedback_Rating	int	Yes	NULL

Dumping data for table Feedback

F001	P001	2024-08-06	Great service and staff	5
F002	P002	2024-08-07	Very helpful and caring staff	4
F003	P003	2024-08-08	Quick service, would recommend	5
F004	P004	2024-08-09	Satisfactory treatment	3
F005	P005	2024-08-10	Good overall experience	4
F006	P006	2024-08-11	Needs improvement in waiting time	2
F007	P007	2024-08-12	Professional and friendly service	5
F008	P008	2024-08-13	Had a great experience	4
F009	P009	2024-08-14	Okay service, but can be better	3
F010	P010	2024-08-15	I would definitely return	5

Table structure for table HODs

Column	Type	Null	Default
Doc_ID	varchar(10)	No	
Dept_ID	varchar(10)	No	

Dumping data for table HODs

D001	D01
D002	D02
D003	D03
D004	D04
D005	D05
D006	D06
D007	D07
D008	D08
D009	D09
D010	D10

Table structure for table In_Patients

Column	Type	Null	Default
In_Patient_ID	varchar(10)	No	
Pat_ID	varchar(10)	Yes	NULL
Room_ID	int	Yes	NULL
Admission_Date	date	Yes	NULL
Discharge_Date	date	Yes	NULL

Dumping data for table In_Patients

IP001	P001	102	2024-07-01	2024-07-10
IP002	P002	103	2024-07-02	2024-07-12
IP003	P003	101	2024-07-03	2024-07-13
IP004	P004	104	2024-07-04	2024-07-14
IP005	P005	105	2024-07-05	2024-07-15

Table structure for table Medicines

Column	Type	Null	Default
Med_ID	varchar(10)	No	
Med_Name	varchar(100)	Yes	NULL
Med_company	varchar(100)	Yes	NULL
Med_price	decimal(10,2)	Yes	NULL
Med_dose	varchar(50)	Yes	NULL
Med_type	varchar(50)	Yes	NULL

Dumping data for table Medicines

M001	Paracetamol	Nigerian Pharma	150.00	500mg	Tablet
M002	Ibuprofen	Health Co.	200.00	400mg	Tablet
M003	Amoxicillin	MediCare	300.00	250mg	Capsule
M004	Metformin	Drug Makers	250.00	500mg	Tablet
M005	Ciprofloxacin	Best Meds	350.00	500mg	Tablet
M006	Syrup	CureCo	180.00	250mg	Syrup
M007	Azithromycin	PharmaPlus	400.00	500mg	Tablet
M008	Lisinopril	MedPlus	320.00	10mg	Tablet
M009	Simvastatin	HealthAid	450.00	20mg	Tablet
M010	Insulin	DiabetCare	600.00	100ml	Injection

Table structure for table Out_patients

Column	Type	Null	Default
Out_Patient_ID	varchar(10)	No	

Pat_ID	varchar(10)	Yes	NULL
Visit_Date	date	Yes	NULL
Treatment_Provided	varchar(255)	Yes	NULL

Dumping data for table Out_patients

OP006	P006	2024-07-16	Follow-up for heart health
OP007	P007	2024-07-17	Medication for hypertension
OP008	P008	2024-07-18	Consultation for knee surgery
OP009	P009	2024-07-19	Follow-up on migraine treatment
OP010	P010	2024-07-20	Dental check-up and cleaning

Table structure for table Patient

Column	Type	Null	Default
Pat_ID	varchar(10)	No	
Pat_fname	varchar(50)	Yes	NULL
Pat_mname	varchar(50)	Yes	NULL
Pat_lname	varchar(50)	Yes	NULL
Pat_gender	char(1)	Yes	NULL
Pat_DOB	date	Yes	NULL
Pat_contactno	bigint	Yes	NULL
Pat_emailID	varchar(100)	Yes	NULL
Pat_Address	varchar(255)	Yes	NULL
Pat_MedHistory	varchar(255)	Yes	NULL
Pat_BloodGroup	varchar(5)	Yes	NULL

Dumping data for table Patient

P001	Chinonso	Emeka	Okafor	M	1990-05-15	8031234567	chinonso.o.okafor@example.com	123 Main St, Lagos	None	O+
P002	Amaka	Nneka	Ibe	F	1985-11-20	8022345678	amaka.ibexample.com	456 Maple Ave, Enugu	Asthma	A-
P003	Ifeoma	Chukwu	Nwank	F	1992-02	8053456	ifeoma.	789 Elm	Diabetes	B+

			wo		-10	789	nwankw o@exa mple.co t m	St, Port Harcour t		
P004	Obinna	Ifeanyi	Obi	M	1988-08 -30	8064567 890	obinna.o bi@exa mple.co m	321 Pine St, Abuja	None	AB+
P005	Uche	Cynthia	Nwosu	F	1995-01 -25	8075678 901	uche.nw osu@ex ample.c om	654 Oak St, Owerri	None	O-
P006	Emeka	Obinna	Chukwu di	M	1989-07 -16	8086789 012	emeka.c hukwudi @exam ple.com	111 Cedar St, Lagos	None	O+
P007	Ngozi	Chidera	Eze	F	1993-03 -09	8097890 123	ngozi.ez e@exa mple.co m	222 Birch St, Enugu	Hyperte nsion	A+
P008	Chukwu emeka	Clement	Okoro	M	1982-12 -12	8108901 234	chukwu emeka.o koro@e xample. com	333 Cedar Rd, Port Harcour t	None	B-
P009	Chinyer e	Ebere	Nwank wo	F	1994-10 -20	8119012 345	chinyere .nwank wo@ex ample.c om	444 Elm Ave, Abuja	None	AB-
P010	Ifeanyi	Obinna	Okwu	M	1981-04 -27	8120123 456	ifeanyi. okwu@ example .com	555 Maple St, Owerri	Diabetes	O-

Table structure for table Rooms

Column	Type	Null	Default
Room_ID	int	No	
Room_Type	varchar(50)	Yes	NULL
Room_Capacity	int	Yes	NULL

Room_Charges	decimal(10,2)	Yes	NULL
--------------	---------------	-----	------

Dumping data for table Rooms

101	General Ward	4	3000.00
102	Private Room	1	7000.00
103	Intensive Care Unit	2	15000.00
104	Surgery Room	1	10000.00
105	Pediatric Ward	6	3500.00
106	Maternity Ward	3	4000.00
107	Isolation Room	1	8000.00
108	Rehabilitation Room	2	5000.00
109	Dialysis Unit	3	6000.00
110	Pharmacy	2	0.00
111	Admin Area	5	0.00

Table structure for table Staff

Column	Type	Null	Default
Staff_ID	varchar(10)	No	
Dept_ID	varchar(6)	No	
Staff_fname	varchar(40)	No	
Staff_mname	varchar(40)	Yes	NULL
Staff_lname	varchar(40)	Yes	NULL
Staff_gender	varchar(2)	No	
Staff_contactno	bigint	No	
Staff_type	varchar(30)	Yes	NULL
Staff_roomassigned1	int	No	
Staff_roomassigned2	int	Yes	NULL
Staff_dutystarttime	time	No	
Staff_dutyendtime	time	No	
Staff_holiday	varchar(10)	Yes	NULL
Staff_Charge	int	No	

Dumping data for table Staff

S001	D01	Segun Oluwa Adeba M	7012345678	Nurse	101	102	08:00:00	16:00:00	Saturday	30000
S002	D02	Nimofe Oladunni Ogunlana F	7023456789	Pharmacist	110	NULL	09:00:00	17:00:00	Sunday	60000
S003	D01	Patricia Chinelo Ifeanyi F	8134567890	Lab Technician	103	109	08:00:00	16:00:00	Monday	40000

S004	D02	David	Tunde	Adeye	M	9045678901	Admi	111	NULL	09:00:00	17:00:00	Tuesd	50000
S005	D01	Mary	Nkechi	Okoro	F	9056789012	Nurse	102	105	08:00:00	16:00:00	Wedn	30000
S006	D03	Esther	Oluwa	Ojo	F	8067890123	Recep	111	NULL	08:00:00	16:00:00	Thurs	25000
S007	D03	Joy	Olami	Adetol	F	8178901234	Physio	107	108	08:00:00	16:00:00	Friday	35000
S008	D04	Kingsl	Chijio	Iwuan	M	8189012345	Techni	109	111	09:00:00	17:00:00	Saturd	40000
S009	D05	Fiyin	Temit	Oguny	F	8090123456	Clean	106	107	07:00:00	15:00:00	Sunda	20000
S010	D05	Daniel	Chijok	Obi	M	9101234567	Securi	111	NULL	18:00:00	06:00:00	Mond	28000

Table structure for table Test

Column	Type	Null	Default
Test_ID	varchar(10)	No	
Test_Name	varchar(100)	Yes	NULL
Test_Prereq	varchar(100)	Yes	NULL

Dumping data for table Test

T01	Blood Test	None
T02	X-Ray	None
T03	MRI Scan	Neurological symptoms
T04	CT Scan	Headache
T05	Urine Test	None
T06	Lipid Profile	None
T07	CT Angiography	Chest pain
T08	Chest X-Ray	Cough
T09	Electrocardiogram	Chest pain
T10	Thyroid Function Test	Thyroid symptoms

Table structure for table Treatment

Column	Type	Null	Default
Visit_ID	varchar(10)	No	
App_ID	varchar(10)	No	
Test_ID	varchar(10)	No	

Test_Result	tinyint(1)	Yes	NULL
Current_Treatment	varchar(255)	Yes	NULL
Extra_Comment	varchar(255)	Yes	NULL

Dumping data for table Treatment

V001	A001	T01	1	General check-up	NULL
V002	A002	T01	1	Asthma control	Prescribed inhaler
V003	A003	T02	0	Knee pain treatment	Recommended physiotherapy
V004	A004	T03	1	Migraine treatment	Advised rest and hydration
V005	A005	T01	1	General health advice	Maintain a balanced diet
V006	A006	T01	1	Regular heart check	Lifestyle changes advised
V007	A007	T04	0	Blood pressure control	Medication prescribed
V008	A008	T02	1	Knee surgery planned	Consultation scheduled
V009	A009	T03	1	Migraine management	Further tests advised
V010	A010	T01	1	Dental care advised	Regular check-ups recommended

Table structure for table Visit

Column	Type	Null	Default
Visit_ID	varchar(10)	No	
App_ID	varchar(10)	Yes	NULL
Pat_weight	int	Yes	NULL
Pat_height	varchar(5)	Yes	NULL
Disease_name	varchar(50)	Yes	NULL
Visit_charge	int	Yes	NULL

Dumping data for table Visit

V001	A001	70	5.9	None	2000
V002	A002	60	5.5	Asthma	1500

V003	A003	80	6.0	Knee Pain	2500
V004	A004	75	5.8	Migraine	2200
V005	A005	65	5.7	None	1800
V006	A006	90	5.10	None	2100
V007	A007	68	5.4	Hypertension	1700
V008	A008	85	6.1	Knee Surgery	2600
V009	A009	72	5.6	Migraine	2300
V010	A010	78	5.8	Dental check-up	2000

5.0 Data Extraction

Data extraction is critical to any database management system, enabling users to retrieve relevant information efficiently. In the Hospital Database Management System (HMDB), SQL (Structured Query Language) queries are employed to extract data from various tables, allowing healthcare professionals and administrators to access the information necessary for decision-making and patient care. Here, we outline several examples of SQL queries used for data extraction in the HMDB, illustrating how these queries facilitate access to vital information.

5.1 Extracting Patient Information

To retrieve comprehensive information about patients, including their personal details and medical history, the following SQL query can be employed:

```
SELECT Pat_ID, Pat_fname, Pat_mname, Pat_lname, Pat_gender, Pat_DOB, Pat_contactno,  
Pat_emailID, Pat_Address, Pat_MedHistory, Pat_BloodGroup  
FROM Patient;
```

This query selects all relevant attributes from the `Patient\_personal\_info` table, providing an overview of each patient's identity and medical background. It serves as a foundational query for understanding patient demographics and medical histories.

5.2 Retrieving Appointment Details

To extract a list of appointments along with associated patient and doctor information, the following SQL query can be used:

```
SELECT A.App_ID, A.App_Date_Time, A.App_Description, P.Pat_fname, P.Pat_lname,  
D.Doc_fname, D.Doc_lname  
FROM Appointment A  
JOIN Patient P ON A.Pat_ID = P.Pat_ID  
JOIN Doctor D ON A.Doc_ID = D.Doc_ID;
```

This query joins the `Appointment`, `Patient\_personal\_info`, and `Doctor\_info` tables to present a detailed view of appointments, including the date and time, description, and names of the patient and the doctor involved. This information is crucial for managing schedules and understanding patient interactions with healthcare providers.

5.3 Accessing Doctor and Department Information

To retrieve information about doctors and their respective departments, the following query can be executed:

```
SELECT D.Doc_ID, D.Doc_fname, D.Doc_lname, D.Doc_Specialist, Dept.Dept_name  
FROM Doctor D  
JOIN HODs H ON D.Doc_ID = H.Doc_ID  
JOIN Department Dept ON D.Doc_DeptID = Dept.Dept_ID;
```

This query connects the `Doctor\_info`, `HODs`, and `Department` tables to view each doctor's speciality and department. This data can help in staffing decisions and referral processes within the hospital.

5.4 Extracting Visit Information

To obtain details of patient visits, including weight, height, and disease names, the following SQL query can be utilized:

```
SELECT V.Visit_ID, V.Pat_weight, V.Pat_height, V.Disease_name, A.App_Date_Time,  
P.Pat_fname, P.Pat_lname  
FROM Visit V  
JOIN Appointment A ON V.App_ID = A.App_ID  
JOIN Patient P ON A.Pat_ID = P.Pat_ID;
```

This query combines information from the `Visit`, `Appointment`, and `Patient\_personal\_info` tables to present a comprehensive overview of patient visits, enhancing understanding of patient conditions and treatment effectiveness.

5.5 Viewing Billing Information

To extract billing information associated with patient visits, the following query can be executed:

```
SELECT B.Bill_ID, B.Pat_ID, B.Visit_ID, B.App_ID, B.Doc_ID, B.No_of_Days,  
B.Health_Card, P.Pat_fname, P.Pat_lname  
FROM Billing_info B  
JOIN Patient P ON B.Pat_ID = P.Pat_ID;
```

This query retrieves relevant billing details from the `Billing\_info` table and patient names from the `Patient\_personal\_info` table. This information is essential for financial management and patient billing processes.

5.6 Extracting Medicines Information

To retrieve a list of medicines available in the hospital, along with their respective prices and types, the following SQL query can be employed:

```
SELECT Med_ID, Med_Name, Med_company, Med_price, Med_type  
FROM Medicines;
```

This query selects all attributes from the `Medicines` table, providing a comprehensive inventory of available medicines, their manufacturers, and pricing details. This information is vital for both inventory management and treatment planning.

The ability to extract and analyze data effectively supports informed decision-making within the healthcare environment, ultimately leading to improved patient outcomes and operational efficiency. These queries not only enhance accessibility to data but also facilitate the management of complex workflows inherent in hospital administration.

6.0 Normalization Process

Normalization is a systematic approach to organizing data in a database to minimize redundancy and ensure data integrity. The normalization process consists of several stages, each building upon the previous one to refine the database structure further. In the Hospital Database Management System (HMDB), the normalization process involves four key steps: First Normal Form (1NF), Second Normal Form (2NF), Third Normal Form (3NF), and Boyce-Codd Normal Form (BCNF). Each step enhances the database's design by ensuring it adheres to specific criteria that promote efficiency and consistency in data management.

6.1 First Normal Form (1NF)

To ensure your SQL code adheres to the First Normal Form (1NF), we need to evaluate the structure of each table for atomicity and uniqueness.

Key Points for 1NF

1. **Atomicity:** Each column should contain atomic values (no repeating groups or arrays).
2. **Uniqueness:** Each row should be unique, which you've ensured with primary keys.

Evaluation of Tables

1. Patient Table

Columns:

``Pat_MedHistory``: This column might hold multiple medical conditions (e.g., "Hypertension; Diabetes"), which violates 1NF.

Correction:

Create a separate ``MedicalHistory`` table to store individual medical conditions.

2. Department Table

Evaluation: This table is fine as it is. It has atomic values, and the ``Dept_ID`` serves as a unique identifier.

3. Doctor Table

Evaluation: All fields are atomic, and the table has a unique primary key ``Doc_ID``. This table adheres to 1NF.

4. HODs Table

Evaluation: This table properly combines ``Doc_ID`` and ``Dept_ID`` as a composite key. It adheres to 1NF.

5. Appointment Table

Evaluation: All fields are atomic, and the foreign key relationships are correctly defined. This table adheres to 1NF.

6. Visit Table

Evaluation: The ``Disease_name`` field is atomic. The table adheres to 1NF.

7. Medicines Table

Evaluation: All fields are atomic, and `Med\_ID` serves as a unique identifier. This table adheres to 1NF.

8. Test Table

Evaluation: This table is properly structured with atomic values. It adheres to 1NF.

9. Treatment Table

Evaluation: All fields are atomic, and the composite key `Visit\_ID`, `App\_ID`, and `Test\_ID` ensure uniqueness. This table adheres to 1NF.

10. Rooms Table

Evaluation: The structure is correct, and all values are atomic. This table adheres to 1NF.

11. Staff Table

Problems:

The `Staff\_type` might be problematic if it has multiple roles or descriptions.

The semicolon in `Staff\_Charge INT NOT NULL;` is incorrect syntax.

`Staff\_roomassigned1` and `Staff\_roomassigned2` should reference `Room\_ID` as `INT`, not `VARCHAR`.

Corrections:

Create a separate `StaffType` table if necessary and correct the syntax:

12. In_Patients Table

Evaluation: All fields are atomic and properly reference `Pat\_ID` and `Room\_ID`. This table adheres to 1NF.

13. Out_patients Table

Evaluation: All fields are atomic. This table adheres to 1NF.

14. Billing_info Table

Evaluation: All fields are atomic. This table adheres to 1NF.

15. Feedback Table

Evaluation: The `Feedback\_Rating` field correctly uses a CHECK constraint. This table adheres to 1NF.

Table structure for table MedicalHistory

Column	Type	Null	Default
History_ID	varchar(10)	No	
Pat_ID	varchar(10)	Yes	NULL
Conditions	varchar(255)	Yes	NULL

Table structure for table Patient

Column	Type	Null	Default
Pat_ID	varchar(10)	No	
Pat_fname	varchar(50)	Yes	NULL
Pat_mname	varchar(50)	Yes	NULL
Pat_lname	varchar(50)	Yes	NULL

Pat_gender	char(1)	Yes	NULL
Pat_DOB	date	Yes	NULL
Pat_contactno	bigint	Yes	NULL
Pat_emailID	varchar(100)	Yes	NULL
Pat_Address	varchar(255)	Yes	NULL
Pat_BloodGroup	varchar(5)	Yes	NULL

Table structure for table Staff

Column	Type	Null	Default
Staff_ID	varchar(10)	No	
Dept_ID	varchar(6)	No	
Staff_fname	varchar(40)	No	
Staff_mname	varchar(40)	Yes	NULL
Staff_lname	varchar(40)	Yes	NULL
Staff_gender	char(1)	No	
Staff_contactno	bigint	No	
Staff_type	varchar(30)	Yes	NULL
Staff_roomassigned1	int	No	
Staff_roomassigned2	int	Yes	NULL
Staff_dutystarttime	time	No	
Staff_dutyendtime	time	No	
Staff_holiday	varchar(10)	Yes	NULL
Staff_Charge	int	No	

Table structure for table StaffType

Column	Type	Null	Default
Type_ID	varchar(10)	No	
Type_Description	varchar(50)	Yes	NULL

6.2 Second Normal Form (2NF)

To ensure your SQL schema adheres to the Second Normal Form (2NF), we need to evaluate the changes made for compliance with 1NF, specifically focusing on partial dependencies. A table is in 2NF if:

1. It is in 1NF.
2. All non-key attributes functionally depend on the primary key.

Evaluation of Tables for 2NF

1. Patient Table

Evaluation: There are no partial dependencies since all non-key attributes depend on the full primary key.

2. Department Table

Evaluation: There are no partial dependencies.

3. Doctor Table

Evaluation: All non-key attributes depend on the full primary key.

4. HODs Table

Evaluation: There are no non-key attributes to assess for partial dependencies.

5. Appointment Table

Evaluation: No partial dependencies exist; all non-key attributes depend on the full primary key.

6. Visit Table

Evaluation: There are no partial dependencies since `App\_ID` is a foreign key reference and does not affect the dependence of other attributes.

7. Medicines Table

Evaluation: There are no partial dependencies.

8. Test Table

Evaluation: There are no partial dependencies.

9. Treatment Table

Evaluation: Each non-key attribute depends on the full composite key.

10. Rooms Table

Evaluation: There are no partial dependencies.

11. Staff Table

Evaluation: The `Dept\_ID` is required for identification of the department, while the other attributes depend on `Staff\_ID`.

Solution: Split the `Staff` table into `Staff` and `Department\_Staff`.

12. In_Patients Table

Evaluation: All non-key attributes depend on the full primary key.

13. Out_patients Table

Evaluation: All non-key attributes depend on the full primary key.

14. Billing_info Table

Evaluation: All non-key attributes depend on the full primary key.

15. Feedback Table

Evaluation: All non-key attributes depend on the full primary key.

16. MedicalHistory Table

Evaluation: All non-key attributes depend on the full primary key.

Table structure for table Department_Staff

Column	Type	Null	Default
Dept_ID	varchar(6)	No	
Staff_ID	varchar(10)	No	

Table structure for table Staff

Column	Type	Null	Default
Staff_ID	varchar(10)	No	
Staff_fname	varchar(40)	No	
Staff_mname	varchar(40)	Yes	NULL
Staff_lname	varchar(40)	Yes	NULL
Staff_gender	char(1)	No	
Staff_contactno	bigint	No	
Staff_type	varchar(30)	Yes	NULL
Staff_roomassigned1	int	No	
Staff_roomassigned2	int	Yes	NULL
Staff_dutystarttime	time	No	
Staff_dutyendtime	time	No	
Staff_holiday	varchar(10)	Yes	NULL
Staff_Charge	int	No	

6.3 Third Normal Form (3NF)

To ensure your SQL schema adheres to the Third Normal Form (3NF), we need to evaluate the changes made for compliance with 2NF, specifically focusing on transitive dependencies. A table is in 3NF if:

1. It is in 2NF.
2. No transitive dependency exists; non-key attributes do not depend on other non-key attributes.

Evaluation of Tables for 3NF

1. Patient Table

Evaluation: There are no transitive dependencies; all non-key attributes depend directly on the primary key.

2. Department Table

Evaluation: There are no transitive dependencies.

3. Doctor Table

Evaluation:

``Doc_DeptID` → `Doc_DeptID`` relates to ``Dept_name``.

``Doc_charge`` may depend on ``Doc_Specialist`` (if different specialists have different charge rates).

Correction: Split ``Doc_Specialist`` and ``Doc_charge`` into a separate ``Specialist`` table to eliminate transitive dependencies.

4. HODs Table

Evaluation: There are no non-key attributes to assess for transitive dependencies.

5. Appointment Table

Evaluation: There are no transitive dependencies; all non-key attributes depend directly on the primary key.

6. Visit Table

Evaluation: There are no transitive dependencies; all non-key attributes depend directly on the primary key.

7. Medicines Table

Evaluation: There are no transitive dependencies; all non-key attributes depend directly on the primary key.

8. Test Table

Evaluation: There are no transitive dependencies; all non-key attributes depend directly on the primary key.

9. Treatment Table

Evaluation: There are no transitive dependencies; all non-key attributes depend directly on the composite key.

10. Rooms Table

Evaluation: There are no transitive dependencies; all non-key attributes depend directly on the primary key.

11. Staff Table

Evaluation: There are no transitive dependencies; all non-key attributes depend directly on the composite key.

12. Department_Staff Table

Evaluation: There are no non-key attributes to assess for transitive dependencies.

13. In_Patients Table

Evaluation: There are no transitive dependencies; all non-key attributes depend directly on the primary key.

14. Out_patients Table

Evaluation: There are no transitive dependencies; all non-key attributes depend directly on the primary key.

15. Billing_info Table

Evaluation: There are no transitive dependencies; all non-key attributes depend directly on the primary key.

16. Feedback Table

Evaluation: There are no transitive dependencies; all non-key attributes depend directly on the primary key.

17. MedicalHistory Table

Evaluation: There are no transitive dependencies; all non-key attributes depend directly on the primary key.

Table structure for table Doctor

Column	Type	Null	Default
Doc_ID	varchar(10)	No	
Doc_fname	varchar(50)	Yes	NULL
Doc_mname	varchar(50)	Yes	NULL
Doc_lname	varchar(50)	Yes	NULL
Doc_gender	char(1)	Yes	NULL

Doc_DOB	date	Yes	NULL
Doc_contactno	bigint	Yes	NULL
Doc_emailID	varchar(100)	Yes	NULL
Doc_Address	varchar(255)	Yes	NULL
Specialist_ID	int	Yes	NULL

Table structure for table Specialist

Column	Type	Null	Default
Specialist_ID	int	No	
Specialist_Name	varchar(50)	Yes	NULL
Charge	decimal(10,2)	Yes	NULL

6.4 Boyce-Codd Normal Form (BCNF)

To check for Boyce-Codd Normal Form (BCNF), we need to ensure that all functional dependencies in our database schema satisfy the following conditions:

1. The relation must be in BCNF if, for every functional dependency $(X \rightarrow Y)$, (X) is a super key.
2. There should be no non-trivial functional dependencies where the left-hand side is not a super key.

Evaluation of Tables for BCNF

1. Patient Table

The table is in BCNF.

2. Department Table

The table is in BCNF.

3. Specialist Table

The table is in BCNF.

4. Doctor Table

The `Specialist\_ID` can imply charges or other details, so we need to check if `Specialist\_ID` can be a super key. If the same `Specialist\_ID` can have multiple charge rates or details (dependent on different doctors), it can create issues. This table does not meet BCNF due to potential transitive dependencies via `Specialist\_ID`. Separate out the relationship between doctors and specialists into a separate associative table.

5. HODs Table

This table is in BCNF.

6. Appointment Table

The table is in BCNF.

7. Visit Table

The table is in BCNF.

8. Medicines Table

The table is in BCNF.

9. Test Table

The table is in BCNF.

10. Treatment Table

The table is in BCNF.

11. Rooms Table

The table is in BCNF.

12. Staff Table

The table is in BCNF.

13. Department_Staff Table

The table is in BCNF.

14. In_Patients Table

The table is in BCNF.

15. Out_patients Table

The table is in BCNF.

16. Billing_info Table

The table is in BCNF.

17. Feedback Table

The table is in BCNF.

18. MedicalHistory Table

The table is in BCNF.

Table structure for table Doctor

Column	Type	Null	Default
Doc_ID	varchar(10)	No	
Doc_fname	varchar(50)	Yes	NULL
Doc_mname	varchar(50)	Yes	NULL
Doc_lname	varchar(50)	Yes	NULL
Doc_gender	char(1)	Yes	NULL
Doc_DOB	date	Yes	NULL
Doc_contactno	bigint	Yes	NULL
Doc_emailID	varchar(100)	Yes	NULL
Doc_Address	varchar(255)	Yes	NULL

Table structure for table Doctor_Specialist

Column	Type	Null	Default
Doc_ID	varchar(10)	No	
Specialist_ID	int	No	

Table structure for table Specialist

Column	Type	Null	Default
<i>Specialist_ID</i>	int	No	
Specialist_Name	varchar(50)	Yes	NULL
Charge	decimal(10,2)	Yes	NULL

This systematic approach lays the groundwork for a reliable and effective system that supports the diverse workflows within hospital management, ultimately leading to improved patient care and operational efficiency.

7.0 Additional Functionalities

In a comprehensive Hospital Database Management System (HMDB), it is important to implement additional functionalities that enhance data management, integrity, and automation. Stored procedures, functions, and triggers are essential to a robust database design, facilitating complex operations and ensuring consistency across various processes. This section describes these functionalities and provides scripts for their implementation.

1. Stored Procedures

Stored procedures are precompiled collections of SQL statements that can be executed as a single unit. They are beneficial for encapsulating complex business logic and operations, promoting reusability and maintainability. Below are examples of stored procedures implemented in the Hospital HMDB.

a. Procedure to Register a New Patient

This stored procedure allows registering a new patient by inserting their personal information into the `Patient\_personal\_info` table.

```
DELIMITER //
CREATE PROCEDURE RegisterPatient (
    IN p_fname VARCHAR(20),
    IN p_mname VARCHAR(20),
    IN p_lname VARCHAR(20),
    IN p_gender VARCHAR(2),
    IN p_dob DATE,
    IN p_contactno BIGINT,
    IN p_emailID VARCHAR(50),
    IN p_address VARCHAR(250),
    IN p_medHistory VARCHAR(500),
    IN p_bloodGroup VARCHAR(10)
)
BEGIN
    DECLARE new_id INT;

    -- Get the next available Patient ID
    SELECT IFNULL(MAX(Pat_ID), 0) + 1 INTO new_id FROM Patient_personal_info;

    INSERT INTO Patient_personal_info
        (Pat_ID, Pat_fname, Pat_mname, Pat_lname, Pat_gender, Pat_DOB,
         Pat_contactno, Pat_emailID, Pat_Address, Pat_MedHistory, Pat_BloodGroup)
    VALUES
        (new_id, p_fname, p_mname, p_lname, p_gender, p_dob,
         p_contactno, LOWER(p_emailID), p_address, p_medHistory, p_bloodGroup);
END //
```

```
DELIMITER ;
```

b. Procedure to Schedule an Appointment

This stored procedure allows healthcare administrators to schedule an appointment with a specific doctor for a patient.

```
DELIMITER //
```

```
CREATE PROCEDURE ScheduleAppointment (  
    IN p_patID INT,  
    IN p_docID VARCHAR(6),  
    IN p_appDateTime TIMESTAMP,  
    IN p_description VARCHAR(200)  
)  
BEGIN  
    INSERT INTO Appointment (App_ID, Pat_ID, Doc_ID, App_Date_Time, App_Description)  
    VALUES (UUID(), p_patID, p_docID, p_appDateTime, p_description);  
END //  
DELIMITER ;
```

2. Functions

Functions are similar to stored procedures but are designed to return a single value. They can be used in SQL statements, making them ideal for calculations and data transformations.

a. Function to Calculate Total Billing Amount

This function calculates the total billing amount for a patient based on their visit records.

```
-- Function to Calculate Total Billing Amount
```

```
DELIMITER //
```

```
CREATE FUNCTION GetTotalBillingAmount(p_patID VARCHAR(10))  
RETURNS DECIMAL(10, 2)  
DETERMINISTIC  
READS SQL DATA  
BEGIN  
    DECLARE totalAmount DECIMAL(10, 2);  
  
    SELECT SUM(Med_price) + SUM(Room_Charges) INTO totalAmount  
    FROM Billing_info B  
    JOIN Medicines M ON B.Medicine_ID = M.Med_ID  
    WHERE B.Pat_ID = p_patID;  
  
    RETURN IFNULL(totalAmount, 0);  
END //  
DELIMITER ;
```

3. Triggers

Triggers are special types of stored procedures that automatically execute in response to specific events on a particular table, such as inserts, updates, or deletes. They help maintain data integrity and enforce business rules.

a. Trigger to Update Patient Visit Count

This trigger automatically increments a visit count for a patient each time a new visit is recorded in the `Visit` table.

```
ALTER TABLE Visit
ADD COLUMN Pat_ID VARCHAR(10) AFTER App_ID;
```

```
DELIMITER //
CREATE TRIGGER IncrementVisitCount
AFTER INSERT ON Visit
FOR EACH ROW
BEGIN
    UPDATE Patient
    SET Visit_Count = Visit_Count + 1
    WHERE Pat_ID = NEW.Pat_ID;
END //
DELIMITER ;
```

b. Trigger to Prevent Duplicate Appointments

This trigger prevents the scheduling of duplicate appointments for the same patient with the same doctor on the same date.

```
DELIMITER //
CREATE TRIGGER PreventDuplicateAppointment
BEFORE INSERT ON Appointment
FOR EACH ROW
BEGIN
    DECLARE existingCount INT;

    SELECT COUNT(*) INTO existingCount
    FROM Appointment
    WHERE Pat_ID = NEW.Pat_ID AND Doc_ID = NEW.Doc_ID
    AND DATE(App_Date_Time) = DATE(NEW.App_Date_Time);

    IF existingCount > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot schedule duplicate appointment for the same patient and
doctor on the same date.';
    END IF;
```

END //
DELIMITER ;

These additional functionalities automate routine tasks, enforce business rules, and ensure data consistency, leading to a more efficient and reliable database environment. These components improve the system's overall usability by encapsulating complex logic and operations, enabling healthcare professionals to focus on patient care and operational efficiency rather than manual data management tasks. The strategic use of these functionalities contributes to the effectiveness of the hospital's information management and operational workflows, supporting the delivery of quality healthcare services.

8.0 Final Outcomes

The successful implementation of the Hospital Database Management System (HMDB) is a culmination of strategic planning, careful selection of development tools, and overcoming various challenges throughout the project lifecycle.

1. Implementation Environment

The implementation environment plays a crucial role in the development and functionality of the HMDB. The selected environment ensures the system operates efficiently and is accessible to users across different platforms. The system was developed and tested primarily on Linux Mint. These operating systems provided a stable and user-friendly interface for the development team, facilitating smooth integration of development tools and server management.

2. Development Tools

The selection of appropriate development tools was vital to successfully implementing the HMDB. PhpMyAdmin, the web-based database management tool facilitated the MySQL database management. PhpMyAdmin provided an intuitive interface for executing SQL queries, managing database tables, and performing data manipulation tasks. Its graphical interface simplified complex database operations and allowed the development team to focus on functionality rather than intricate database management tasks. DB Beaver was used to generate the ER Diagram. As the database management system, MySQL was chosen for its performance, security features, and ability to handle large datasets efficiently. The relational nature of MySQL allowed for effective management of the complex relationships between various entities within the HMDB.

3. Challenges Faced

We encountered several challenges throughout the project, each requiring innovative solutions and collaborative efforts to overcome. One significant challenge was ensuring seamless data integration, particularly during the initial phase. The team had to develop custom scripts to clean and normalize the data, ensuring consistency and accuracy before importing it into the new database structure. Also, we faced issues setting up the db viewer so as to visualize the database according.

Conclusion

The development of the Hospital Database Management System (HMDB) represents a significant advancement in the management of healthcare services within a hospital setting. This project aimed to streamline various workflows associated with patient registration, appointment scheduling, and overall data manipulation, thereby enhancing operational efficiency and improving patient care. The systematic approach taken throughout the project's lifecycle, from initial conceptual design to final implementation, ensured the delivery of a robust and user-friendly system tailored to meet the needs of hospital administrators, medical staff, and patients alike. While the HMDB has achieved its primary objectives, there are several potential improvements and enhancements that can be explored to further enrich its capabilities and effectiveness, including integrating the HMDB with other healthcare systems, such as laboratory information systems or electronic health records (EHRs), developing a mobile application for patients could enhance accessibility and engagement, implementing advanced reporting and analytics capabilities would enable hospital administrators to gain insights into operational efficiency, patient outcomes, and resource utilization, and incorporating telemedicine features would allow patients to consult with doctors remotely, providing an essential service in situations where in-person visits are not feasible.

Future enhancements could include leveraging data analytics and machine learning algorithms to predict patient outcomes, identify trends, and improve overall healthcare delivery. These technologies could support proactive healthcare management and personalized patient care. The Hospital Database Management System stands as a transformative solution in healthcare management. Through careful planning and execution, the project has successfully addressed the critical needs of hospital operations, improving efficiency, data management, and patient care. The insights gained from this project not only contribute to current healthcare practices but also provide a platform for future innovations and enhancements that can further elevate the quality of care delivered in hospital settings. The potential for continuous improvement and adaptation in response to evolving healthcare needs ensures that the HMDB will remain a valuable asset in the quest for better healthcare outcomes.